

Does `/init` Help?

A Meta-Study of Repository Context Files for Coding Agents

Marcel Petrick · Claude (Opus 5, revised with Opus 5.5)

19 September 2026 · revised 24 September 2026

Abstract. Coding agents ship a command that writes a repository context file—`CLAUDE.md` or `AGENTS.md`. The agent then loads that file into every later session. Does it help? We read the six controlled studies, the mechanism literature around them, and the prompt we extracted from a shipped Claude Code binary.

Every full-benchmark effect on task success is smaller than the noise. At temperature zero, $\approx 9\%$ of outcomes flip between byte-identical runs. Most published estimates come from designs that ran each task once, so those numbers settle nothing. The largest study, on Claude Code, finds random rules scoring exactly as well as curated ones. Designs that repeat their trials mostly report nulls. One does not: guidance *tuned against an agent’s own failures* raised resolve rate from 25.5 to 33.0% on an open 35B model ($p < 0.001$). It helped the agent reach the right file. It did not improve the patches.

Cost, by contrast, moves reliably: $+20$ to $+23\%$. Splitting the file in two explains the disagreement. Agents follow *instructions*—they invoke a named tool $1.6\times$ per task, against $<0.01\times$ when it goes unmentioned. Agents gain nothing from *repository overviews*, because prose about code answers 4/45 behavioural questions where the code answers 27/45. That gap belongs to the representation, not to the writer. One principle predicts both halves: **scaffolding decays when it substitutes for a capability the model now has, and persists when it supplies what the model cannot derive or enforces what it cannot infer.**

1 Introduction

The practice is near-universal and the evidence is thin. Eleven major harnesses ship an instruction-file mechanism: Claude Code, Codex, Copilot, Cursor, Aider, Gemini CLI, Windsurf, Cline, Amp, Devin and Junie. Almost none of their vendors has published a controlled measurement of whether it works. The whole causal base is six 2026 preprints. None has been replicated, and none has been rebutted.

We ask three questions. Does a context file improve task success? What does it cost? And if the answers are “not measurably” and “20%,” why does the feature exist?

2 Method

We ran seven rounds of literature search, most of them across parallel agents. We also produced two primary sources ourselves. First, we mined the shipped Claude Code binary v2.1.278 and recovered the literal `/init` prompt, an opt-in successor that is off by default, and the exact loading semantics. Second, we measured 35 instruction files in a working repository corpus (median $\approx 1,590$ tokens, p90 $\approx 3,300$).

We did not trust the agents. We re-fetched every load-bearing claim from its primary source, and that caught five errors. Two passes reported incompatible figures from one paper. One cited p -values the paper never reports. Two statistics in wide circulation turned out to be fabricated, and one vendor figure is missing from its own source. A verification log and a self-review of our first draft accompany this paper. Our prose meets measured clear-writing targets—median sentence 13 words, 4.8% over 30, 6.7 passives per 100 sentences, Flesch 47—which tools/readability.py recomputes.

3 Results

3.1 Most point estimates are not measurements

Figure 1 plots every measured effect on task success against the measured run-to-run noise floor. Sam-Bodden [2] froze his protocol before collecting data. He reports that temperature-0 inference “flips $\approx 9\%$ of per-instance outcomes between byte-identical runs,” and calls this “a noise floor under every small effect reported on this benchmark.” Every

Table 1: The complete causal evidence base. All six are 2026 preprints or workshop papers; none is replicated or formally rebutted. “Repeats” denotes multiple runs of the same condition—the column that decides whether a point estimate is a measurement.

Study	Scale	Repeats	Outcome
Gloaguen [1]	438 instances, 4 agents	No (1 $\times$)	No success gain; $+20$ – 23% cost
Khatr [3]	288 runs, 17 tasks	Yes (3 $\times$)	No effect; but MDE ≈ 30 pp
Lulla [4]	124 PRs, 10 repos	No (paired)	-28.6% runtime; correctness spot-checked
McMillan [5]	1,650 sessions	Yes (50 $\times$)	Affirmative null on file structure
Shepard [8]	4 trials, Qwen3.5-35B	Yes (4 $\times$)	$+7.5$ pp, $p < 0.001$; <i>tuned</i> guidance only
Zhang [6]	58 of 500 tasks, Opus 4.6	No (1 $\times$)	$+7$ – 14 pp, none significant; random rules = curated

full-benchmark estimate falls inside it. The study contributing several of them sampled each instance once, with no significance testing.

Read the band for what it is. $\approx 9\%$ is a *per-instance flip rate*—what one run establishes about one instance—not a minimum detectable effect. Repeated trials shrink the standard error of the mean, so they can resolve a smaller difference. That is why the one positive result below survives inside the band. The floor disqualifies the single-run estimate, not the practice.

Single runs correlate only $\rho = 0.417$ with true repeated-rollout reliability [13], and run-level pass rates overstate retry-free coverage by up to 17.8 pp [14]. In one practitioner test, a single run showed `AGENTS.md` 44% *slower* on a task where the five-run median showed it 9–10% *faster*.

3.2 Cost is the one robust effect

Gloaguen et al. [1] find inference cost rises 20% (SWE-bench Lite) to 23% (AGENTbench), with 2–4 additional reasoning

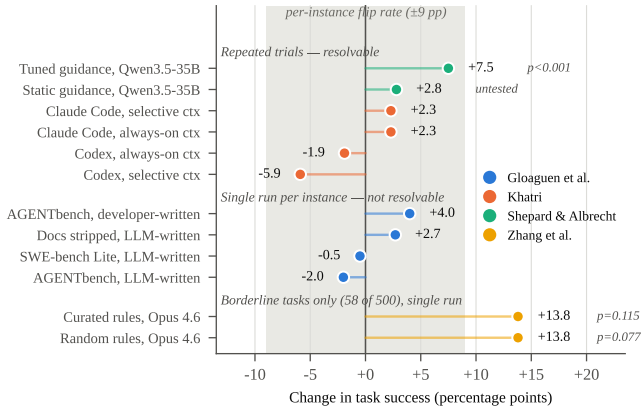


Figure 1: Every measured change in task success, grouped by whether the design can resolve one. The band is the ± 9 pp per-instance flip rate [2]. Of the six from designs with repeats, four are bounded nulls, one (+2.8 pp) is never tested, and one (+7.5 pp, $p < 0.001$) is real. The bottom pair covers only tasks the agent solves some of the time; over all 500 it shrinks to ≈ 1.6 pp (our arithmetic).

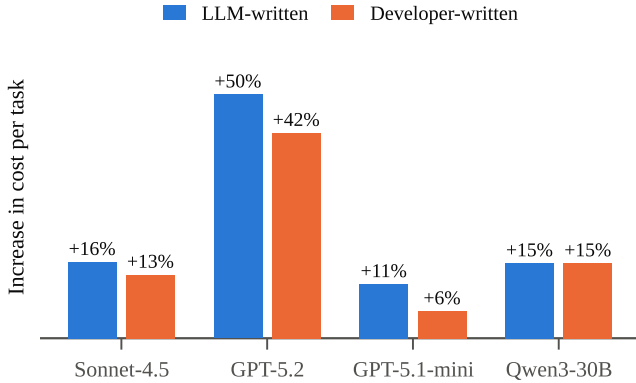


Figure 2: Increase in cost per task on AGENTbench when a context file is present, by model and provenance. Derived from Table 2 of [1]. Unlike the success-rate effects, this is consistent in sign across all four models.

steps, across every model tested (Figure 2). Lulla et al. [4] report the opposite sign (-28.6% runtime) on small PRs, with correctness spot-checked on only 50 of 124.

Cost in tokens is not cost in currency. CLAUDE.md occupies Claude Code’s cached project-context layer; at published rates a 10,000-token file costs \$0.12 across a 50-turn session, and the cached/uncached ratio is exactly $50/6.15 = 8.13\times$, independent of file size. *The monetary argument against a large context file does not survive contact with the arithmetic.*

3.3 Two artefacts, opposite signs

The decomposition in [1] resolves the field’s apparent contradiction: “instructions in the context files are well followed by coding agents, repository overviews ... are not helpful.” Instructions move behaviour—a tool named in the file is invoked $1.6\times$ per instance versus $< 0.01\times$ unmentioned; repository-specific tools $2.5\times$ versus $< 0.05\times$. Overviews do not measurably reduce steps-to-first-relevant-file, despite appearing in 100% of LLM-generated files.

Sam-Bodden [2] supplies the mechanism and forecloses the obvious remedy: prose summaries answer 4/45 behavioural questions about a file where the source answers 27/45, and “the gap belongs to the representation, not the summarizer—a frontier model’s summaries score exactly

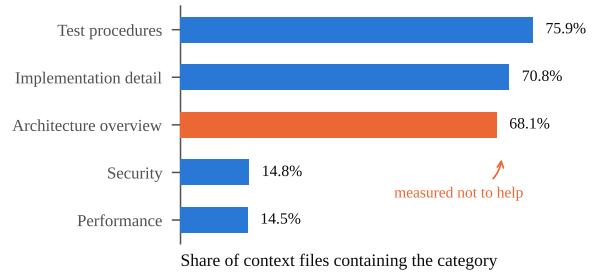


Figure 3: Content categories across 2,303 context files [12]. Architecture overviews—the one category with direct evidence of no benefit [1, 2]—appear in 68.1% of files.

as poorly as a 3B model’s.” A registered hypothesis that structured skeletons would help *failed* ($N=70$, McNemar $p=0.75$). Figure 3 shows that the category measured not to help is present in roughly two-thirds of real files.

The largest study, over 5,000 Claude Code runs on Opus 4.6 [6], adds a twist. On the 58 tasks the agent solves only sometimes, all seven rule files beat no file by 6.9–13.8 pp, yet random rules tie curated ones at 63.8%, and no single contrast is significant. The authors read this as context priming: presence nudges, content does not decide. The only rules that helped alone were prohibitions such as “do not refactor unrelated code.”

3.4 The exception: tuned guidance on a weaker model

One design breaks the pattern, and it is the most careful of the six. Shepard and Albrecht [8] introduce *probe-and-refine tuning*: synthetic bug-fix probes iteratively diagnose and patch a repository’s guidance file through single-shot calls, with no agent loop during tuning. Over four independent trials on SWE-bench Verified with Qwen3.5-35B-A3B, resolve rate runs 25.5% unguided, 28.3% with a static knowledge base, and **33.0%** tuned ($p < 0.001$ for both tuned contrasts). They conclude that “how the guidance is produced is the decisive variable.”

Three things keep this compatible with the nulls. First, the model is an open 35B, not a frontier agent. Their cross-model check also finds the procedure *degrading* on a model too weak to emit diagnostic output, so the scaffolding has a capability window on both sides. Second, the gain came from provenance, not content. The static knowledge base—the condition closest to what `/init` writes—gained 2.8 pp, which the paper does not significance-test. Only the file tuned against observed failures moved the result, which makes it a defect log rather than a repository tour. Third, the improvement has a location. Coverage rises 14.5 pp while per-patch precision stays flat ($\approx 59\%$, $p = 0.119$). Guidance “helps agents reach the correct file rather than improving the quality of the changes they make”—the nulls’ conclusion, reached from the other side.

3.5 The artefact has no brake

Across 247,694 instruction lifetimes in 1,867 repositories, context files grow **+226%** over their lifetime, a net +4.9 instructions per commit, and the older an instruction becomes the *less* likely it is ever deleted [10]. Appending is cheap; verifying a deletion is safe is $O(2^{|D|})$. Independently, 23.0% of such files already contain stale code references [11], and stale lines do harm: agents follow the file over the code, “so a stale convention file costs more than no file” [7]. A second

study of 1,540 rule-evolution events finds the same ratchet. Expansions (29.17%) and enrichments (26.59%) dominate, and developers say they fix agent errors “by adding new negative constraints rather than editing existing ones” [9]. That study also supplies the counterweight: after a rule update, artefact compliance rises from 49.14% to 72.13%. Four structural properties—size, position, architecture and internal contradiction—show *affirmative* null effects on adherence ($BF_{10} = 0.096, 0.053$). One thing does degrade compliance: session length, at -5.6% odds per generated function [5].

4 Discussion: why the command exists

`/init` shipped in Claude Code’s first public build (2025-02-24). It was not a patch for an observed deficiency. But its performance rationale has eroded as the harness gained cheaper homes for the same content: hooks (2025-06), subagents (2025-07), Skills and a context-saving Explore subagent (2025-10), path-scoped rules (2025-12) and auto-memory (2026-02).

Vendor framing tracks that shift exactly. In 2025-04 the file was “an ideal place” for documentation, with no size limit. By 2026-09 it carries a hard 200-line target and a warning: “bloated CLAUDE.md files cause Claude to ignore your actual instructions.” A successor to `/init`, opt-in and still off by default in v2.1.282, states the thesis outright: “CLAUDE.md is loaded into every Claude Code session, so it must be concise—only include what Claude would get wrong without it.”

Three rationales cannot decay, and none is about task success. `/init` migrates Cursor, Copilot, Windsurf, Devin and Cline rule files. It onboards new users to the memory system. And it produces a human-readable artefact for the team.

A principle

Derived from an unrelated literature—chain-of-thought [15], prefill, verification prompting, minimal agent scaffolds [16]—and found to predict the present result:

Scaffolding decays when it substitutes for a capability the model now has natively; it persists when it supplies information the model structurally cannot derive, or enforces a constraint it cannot infer.

A repository overview substitutes for reading the codebase—and the agent can now grep. Commands, gotchas and prohibitions supply what no amount of reading yields. The principle’s sharpest test is §3.4: the same artefact is worth $+7.5$ pp to a 35B model that cannot reliably find the right file and nothing measurable to agents that can.

5 Verdict

The practice is sound; the default artefact is not. Run `/init` as a draft, then delete the architecture tour. Keep only what the repository cannot state about itself, and count imperatives rather than lines. Move enforcement to hooks, procedures to skills, and module detail to path-scoped files. Shrink the file to protect compliance, not to save money.

Above all, record what the agent got *wrong* rather than what the repository contains. That is the one intervention with a significance-tested gain behind it, and the test Anthropic’s own new `/init` applies. On a frontier agent, expect no gain in success rate—they fail on implementation skill, which no document repairs. On a smaller model, weight all of this differently.

Limitations. We rest on six unreplicated preprints, most of them underpowered for the effects they report. We ran no experiment of our own. Our cost tables are modelled rather than measured, and we read one binary version on one machine. Two later sweeps each found a study the earlier rounds had missed; the second, public since April, is the largest of all. Assume the same of this version. The decisive experiment—ablating instructions against overviews under one shared success metric—remains unrun, and it is cheap.

6 Open experiments

The gaps are specific, and several are cheap to close.

1. **Halve the file.** Ablate *instructions only* against *overview only*, *both* and *neither*, under one success metric with repeated runs. Every ingredient exists: the released AGENTbench harness makes it a week of compute. This is the decisive experiment, and nobody has run it.
2. **Measure all three at once.** No study reports success, cost and latency together with repeats. Against a ≈ 9 pp noise floor [2], a design powered for 5 pp needs 120–200 tasks [3]—large, but not exotic.
3. **The capability window.** Guidance pays on a 35B model [8] and not on frontier agents, but no design spans a capability ladder. “Does it help?” is missing its qualifier: help whom? Nobody has run probe-and-refine on a frontier agent either—the cheapest experiment still open.
4. **Generalisation.** The evidence is overwhelmingly Python and TypeScript; the practice may pay in languages the model knows less well [1].

References

- [1] T. Gloaguen, N. Mündler, M. Müller, V. Raychev, M. Vechev. *Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?* MemAgents @ ICLR 2026. arXiv:2602.11988.
- [2] B. Sam-Bodden. *What Context Does a Coding Agent Actually Need to Act?* arXiv:2607.09691.
- [3] P. Khatri. *Do Context Files Help Coding Agents? A Two-Agent Ablation Study on Real Repositories.* arXiv:2607.27250.
- [4] J. L. Lulla et al. *On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents.* JAWs @ ICSE 2026. arXiv:2601.20404.
- [5] D. McMillan. *Instruction Adherence in Coding Agent Configuration Files.* arXiv:2605.10039.
- [6] X. Zhang et al. *Guardrails Beat Guidance: A Large-Scale Study of Rules, Skills, and Persistent Configuration for Coding Agents.* arXiv:2604.11088.
- [7] B. Mohammadi et al. *The Working Set of a Coding Agent: Coherence Debt in Repository-Scale Tasks.* arXiv:2608.16630.
- [8] A. Shepard, J. Albrecht. *Probe-and-Refine Tuning of Repository Guidance for Coding Agents.* arXiv:2606.20512.
- [9] G. Cai, R. Li, P. Liang, Z. Li, M. Shahin. *Rule Taxonomy and Evolution in AI IDEs: A Mining and Survey Study.* arXiv:2606.12231.
- [10] K. Chakrabarti. *Why Does CLAUDE.md Keep Growing? Catastrophic Remembering in Agentic Coding.* arXiv:2608.11095.
- [11] C. Treude, S. Baltes. *Context Rot in AI-Assisted Software Development.* arXiv:2606.09090.
- [12] P. Chatlatanagulchai et al. *Agent READMEs: An Empirical Study of Context Files for Agentic Coding.* arXiv:2511.12884.
- [13] Y. Jiang et al. *Beyond Pass@k.* arXiv:2608.14711.
- [14] *Accuracy, Stability, and Repeated-Run Reliability of LLMs on Deterministic Programming Tasks.* arXiv:2606.00920.
- [15] Z. Sprague et al. *To CoT or not to CoT?* ICLR 2025. arXiv:2409.12183.
- [16] C. Xia et al. *Agentless: Demystifying LLM-based Software Engineering Agents.* arXiv:2407.01489.